Name : T Bhanu Teja

Reg no: 192472259

UNIT III

CONSTRUCTORS, DESTRUCTORS AND OPERATOR OVERLOADING

C++ Programming – Questions with Answers

Syllabus scope: constructors, constructor overloading, copy constructors, dynamic constructors, destructors, deep and shallow copy, unary and binary operator overloading, stream insertion and extraction, and operator overloading rules.

Assessment Answer Format

Section A – Debugging: Each answer identifies the defective line and error category, explains the root cause, provides complete corrected C++ code, and includes a dry run with the predicted output.

Section B – Pseudocode: Each answer includes a brief design overview, structured pseudocode using the prescribed lifecycle and operator syntax, and a trace table using the sample input.

SECTION A: DEBUGGING QUESTIONS

Each debugging question is presented with its topic, problem statement, original buggy code, a brief answer introduction, corrected C++ code, dry run, and exact output.

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

Topic: Copy Constructors, Parameter Passing Mechanisms.

Problem Statement: A DynamicString class allocates a character buffer on the heap. Its copy constructor is incorrectly declared with a pass-by-value parameter, causing the compiler to reject the copy-constructor definition.

Buggy Code:

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char *buffer;
    int length;

public:
    // Parameterized Constructor
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    // Line 17: Copy Constructor defined with Pass-by-Value!
    DynamicString(DynamicString source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};
```

```
int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1; // Intended copy construction
    s2.print();
    return 0;
}
```

ANSWER

A copy constructor must receive the source object by reference. Passing it by value would require copying the argument to initialize the parameter, which would call the same copy constructor again.

1. Line number and error classification / root cause: Line 17 declares DynamicString(DynamicString source). A copy constructor cannot take its own class type by value because initializing the parameter itself requires a copy, recursively invoking the same constructor. This is rejected by the language; it is not a runtime recursion.

2. Corrected signature: DynamicString(const DynamicString &source). The reference avoids making another copy, and const allows copying from const objects and temporary-compatible sources without modifying the original.

3. Corrected C++ code:

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char *buffer;
    int length;

public:
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    DynamicString(const DynamicString &source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
```

```
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
    return 0;
}
```

4. Dry run: s1 allocates a buffer for 15 characters plus the null terminator. Copy construction allocates a separate buffer for s2 and copies the text. Both objects later release their own buffers.

Step	Operation	Result
1	Construct s1	Buffer allocated for "Antigravity C++"
2	Copy construct s2 from s1	New independent buffer allocated and copied
3	s2.print()	Displays Antigravity C++
4	End of main	s2 and s1 destructors each delete their own buffer

4. Exact output:

```
Antigravity C++
```

Question 2: Shallow Copy & Double-Free Runtime Crash

Topic: Dynamic Constructors, Destructors, Shallow vs. Deep Copy.

Problem Statement: AudioBuffer allocates a dynamic array and deletes it in its destructor. The original code omits a custom copy constructor and passes the object by value, so the compiler-generated copy duplicates only the pointer and both objects attempt to delete the same memory.

Buggy Code:

```
#include <iostream>

class AudioBuffer {
public:
   int *samples;
   int capacity;

   AudioBuffer(int cap) : capacity(cap) {
      samples = new int[capacity];
      for (int i = 0; i < capacity; ++i) {
         samples[i] = i * 10;
      }
      std::cout << "Buffer allocated at: " << samples << std::endl;
   }

   // Programmer omitted copy constructor!

   ~AudioBuffer() {
      std::cout << "Deallocating buffer at: " << samples << std::endl;
      delete[] samples; // Crash occurs here during execution!
   }
};

void processAudio(AudioBuffer buf) { // Passed by value!
   std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}

int main() {
   AudioBuffer track(5);
   processAudio(track);
   std::cout << "Back in main, sample 0: " << track.samples[0] << std::endl;
   return 0;
}
```

ANSWER

1. Line number and error classification / root cause: The implicit copy constructor performs a member-wise shallow copy. The copied object and original object point to the same samples array. When the by-value parameter is destroyed, it deletes the shared array; the original object later tries to delete it again, causing undefined behavior and commonly a double-free crash.

2. Corrected approach: Implement a deep copy constructor that allocates a new array and copies every sample. Pass the object by value as the question intends so the separate allocation and deallocation can be observed.

3. Corrected C++ code:

```
#include <iostream>

class AudioBuffer {
public:
    int *samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) {
            samples[i] = i * 10;
        }
        std::cout << "Buffer allocated" << std::endl;
    }

    AudioBuffer(const AudioBuffer &other) {
        capacity = other.capacity;
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) {
            samples[i] = other.samples[i];
        }
        std::cout << "Deep copy allocated" << std::endl;
    }

    ~AudioBuffer() {
        std::cout << "Buffer deallocated" << std::endl;
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: "
              << track.samples[0] << std::endl;
    return 0;
}
```

4. Dry run: track allocates its own five-element array. Passing track by value invokes the deep copy constructor, which allocates a different array. The parameter is destroyed when processAudio returns, but track's array remains valid until main ends.

Step	Object / action	Memory and value
1	Construct track	Array A allocated; samples[0] = 0
2	Call processAudio(track)	Copy constructor allocates separate Array B
3	Inside processAudio	buf.samples[0] = 0
4	Function returns	buf destructor deletes Array B
5	Back in main	track.samples[0] remains 0
6	End of main	track destructor deletes Array A

4. Expected output:

```
Buffer allocated
Deep copy allocated
Processing sample 0: 0
Buffer deallocated
Back in main, sample 0: 0
Buffer deallocated
```

Question 3: Stream Insertion Operator << Member Function Violation

Topic: Defining Operator Overloading, Member vs. Non-Member/Friend.

Problem Statement: Fraction incorrectly defines operator<< as a member. This makes the left operand need to be a Fraction, so std::cout << f1 fails while f1 << std::cout uses the wrong operand order.

Buggy Code:

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}
```

```
    // Line 11: Declaring operator<< as a member function
    std::ostream& operator<<(std::ostream &os) {
        os << numerator << "/" << denominator;
        return os;
    }
};

int main() {
    Fraction f1(3, 4);

    // Line 20: Natural stream syntax fails to compile!
    std::cout << f1 << std::endl;

    // Line 23: Compiles, but inverts standard semantics
    f1 << std::cout;

    return 0;
}
```

ANSWER

1. Line number and error classification / root cause: Line 11 defines operator<< as a Fraction member. A binary member operator call is interpreted as left.operator<<(right). Therefore std::cout << f1 looks for an operator<< member of std::ostream, not Fraction. The expression f1 << std::cout calls the Fraction member but reverses normal stream semantics.

2. Corrected approach: Define operator<< as a non-member friend function. The left operand is std::ostream&, and the right operand is const Fraction&.

3. Corrected C++ code:

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    friend std::ostream& operator<<(std::ostream& os,
                                    const Fraction& f);
```

```
};

std::ostream& operator<<(std::ostream& os, const Fraction& f) {
    os << f.numerator << "/" << f.denominator;
    return os;
}

int main() {
    Fraction f1(3, 4);
    std::cout << f1 << std::endl;
    return 0;
}
```

4. Dry run: f1 contains numerator = 3 and denominator = 4. std::cout is passed as the first argument and f1 as the second. The friend function prints 3/4 and returns the stream to support chaining.

Step	Operation	Result
1	Create Fraction f1(3,4)	numerator=3, denominator=4
2	Call operator<<(cout, f1)	Prints 3/4
3	Return stream reference	Supports further stream output

4. Exact output:

```
3/4
```

Question 4: Prefix vs. Postfix Increment Signature Inversion

Topic: Overloading Unary Operators (Prefix ++, Postfix ++(int)).

Problem Statement: StepCounter has the prefix and postfix increment signatures reversed. Prefix should increment and return the modified object by reference; postfix should save the old value, increment the object, and return the old snapshot by value.

Buggy Code:

```
#include <iostream>

class StepCounter {
private:
    int steps;
```

```
public:
    StepCounter(int s = 0) : steps(s) {}

    // Line 10: Intended as POSTFIX, but missing dummy int parameter
    StepCounter operator++() {
        StepCounter temp = *this;
        steps++;
        return temp;
    }

    // Line 17: Intended as PREFIX, but includes dummy int parameter
    StepCounter& operator++(int) {
        steps++;
        return *this;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);

    // Developer expects prefix (++c1) to increment immediately and return modified object
    StepCounter c2 = ++c1;
    c2.display(); // Prints 10 instead of 11!
    c1.display(); // Prints 11

    return 0;
}
```

ANSWER

1. Line number and error classification / root cause: The operator++() at Line 10 is intended to be postfix but lacks the dummy int parameter. The operator++(int) at Line 17 is intended to be prefix but incorrectly has the dummy parameter. Their return behaviors are also reversed.

2. Correct signatures: Prefix: StepCounter& operator++(). Postfix: StepCounter operator++(int). The dummy int distinguishes postfix syntax; the compiler supplies its value.

3. Corrected C++ code:

```
#include <iostream>
```

```
class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    StepCounter& operator++() {
        ++steps;
        return *this;
    }

    StepCounter operator++(int) {
        StepCounter old = *this;
        steps++;
        return old;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);

    StepCounter c2 = ++c1;
    c2.display();
    c1.display();

    StepCounter c3 = c1++;
    c3.display();
    c1.display();

    return 0;
}
```

4. Dry run: Initially c1 = 10. Prefix increments c1 to 11 and returns c1 by reference, so c2 becomes 11. Postfix saves 11 in c3, increments c1 to 12, and returns the old snapshot.

Step	**Expression**	**c1**	**c2**	**c3**
Initial	—	10	—	—
1	c2 = ++c1	11	11	—
2	c3 = c1++	12	11	11

4. Exact output:

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

Question 5: Violation of Fundamental Operator Overloading Rules

Topic: Rules for Overloading Operators.

Problem Statement: MathVector attempts to overload the member access dot operator, overload + for two built-in integers, and invent a ** exponentiation operator. These are prohibited by C++ operator overloading rules.

Buggy Code:

```
#include <iostream>

class MathVector {
public:
    double x, y;
    MathVector(double a, double b) : x(a), y(b) {}

    // Line 8: Attempting to overload member access dot operator
    bool operator.(const MathVector &other) {
        return (x == other.x && y == other.y);
    }
};

// Line 14: Attempting to overload operator for built-in primitive types
int operator+(int a, int b) {
    return a * b;
}

// Line 19: Attempting to invent a non-existent operator '**' for exponentiation
MathVector operator**(MathVector a, int power) {
    // ...
}

int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);
    return 0;
```

```
}
```

ANSWER

1. Line number and error classification / root cause: Line 8 attempts to overload operator.; the dot operator cannot be overloaded. Line 14 attempts to overload + for two int operands, but at least one operand must be a user-defined type or enumeration. Line 19 invents **, which is not an operator token in C++ and cannot be introduced by programmers.

2. Corrected approach: Use legal operators == and + for MathVector, and use a normal member method power() for exponentiation. The non-overloadable operators include ., .*, ::, ?: and sizeof.

3. Corrected C++ code:

```
#include <iostream>

class MathVector {
public:
    double x, y;

    MathVector(double a = 0, double b = 0) : x(a), y(b) {}

    bool operator==(const MathVector& other) const {
        return (x == other.x && y == other.y);
    }

    MathVector operator+(const MathVector& other) const {
        return MathVector(x + other.x, y + other.y);
    }

    MathVector power(int exponent) const {
        double resultX = 1;
        double resultY = 1;

        for (int i = 0; i < exponent; ++i) {
            resultX *= x;
            resultY *= y;
        }
        return MathVector(resultX, resultY);
    }

    void display() const {
        std::cout << "(" << x << ", " << y << ")" << std::endl;
```

```
    }
};

int main() {
    MathVector v1(2.0, 3.0);
    MathVector v2(2.0, 3.0);

    std::cout << "Equal: " << (v1 == v2) << std::endl;

    MathVector v3 = v1 + v2;
    std::cout << "Sum: ";
    v3.display();

    MathVector v4 = v1.power(2);
    std::cout << "Power: ";
    v4.display();

    return 0;
}
```

4. Dry run: v1 and v2 have identical coordinates, so == returns true. Addition gives (4,6). power(2) squares each coordinate, giving (4,9).

Operation	**Calculation**	**Result**
v1 == v2	(2==2) AND (3==3)	true (1 when printed as bool)
v1 + v2	(2+2, 3+3)	(4, 6)
v1.power(2)	(2^2, 3^2)	(4, 9)

4. Exact output:

```
Equal: 1
Sum: (4, 6)
Power: (4, 9)
```

SECTION B: PSEUDOCODE WRITING QUESTIONS

Each pseudocode question is presented with its topic, scenario, student tasks, structured pseudocode answer, and sample-input trace table.

Question 1: Dynamic String Engine with Deep Copy & Overloaded +

Topic: Dynamic Constructors, Deep Copy Constructors, Destructors, Binary +.

Scenario: Design DynStr to manage heap-allocated character buffers. It must allocate and copy source text, deep-copy another DynStr into a fresh buffer, release its buffer in the destructor, and overload + to concatenate two strings into a new DynStr.

Student Tasks: Write DynStr and its driver. Trace heap allocations, buffer sizes, and character copies for strA = "Hello " and strB = "World!".

ANSWER

1. Design overview: Each object owns its own character buffer. The copy constructor allocates a separate buffer. The + operator creates a concatenated result without sharing either operand's buffer.

2. Structured pseudocode:

```
CLASS DynStr
PRIVATE:
  dataPtr : POINTER TO CHARACTER
  charCount : INTEGER

PUBLIC:
  CONSTRUCTOR(sourceString : STRING)
    charCount <- LENGTH(sourceString)
    dataPtr <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER
    COPY sourceString INTO dataPtr
    dataPtr[charCount] <- NULL
  ENDCONSTRUCTOR

  COPY CONSTRUCTOR(REF other AS DynStr)
    charCount <- other.charCount
    dataPtr <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER
    FOR i <- 0 TO charCount
      dataPtr[i] <- other.dataPtr[i]
    ENDFOR
  ENDCONSTRUCTOR

  DESTRUCTOR()
    DEALLOCATE dataPtr
```

```
    dataPtr <- NULL
  ENDDESTRUCTOR

  OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
    DECLARE combined : STRING
    combined <- CONCATENATE(dataPtr, rhs.dataPtr)
    RETURN DynStr(combined)
  ENDOPERATOR

  METHOD Display()
    DISPLAY dataPtr
  ENDMETHOD
ENDCLASS

BEGIN
  DECLARE strA : DynStr <- DynStr("Hello ")
  DECLARE strB : DynStr <- DynStr("World!")
  DECLARE strC : DynStr
  strC <- strA + strB
  strC.Display()
END
```

3. Dry-run trace: "Hello " has 6 characters and needs 7 bytes including the null terminator. "World!" has 6 characters and needs 7 bytes. The concatenated text has 12 characters and needs 13 bytes.

Object	Text	Character count	Allocated buffer size
strA	Hello	6	7 bytes
strB	World!	6	7 bytes
strC	Hello World!	12	13 bytes

Expected output:

```
Hello World!
```

Question 2: Complex Number System with Constructor Overloading & Operators (+, *, ==)

Topic: Constructor Overloading, Operator Overloading (+, *, ==).

Scenario: Design Complex with default, one-argument and two-argument constructors. Overload + for addition, * for multiplication, and == to compare both components. Include a formatted display method.

Student Tasks: Write the constructors, overloaded operators and helper methods. Trace the product of C1 = 3 + 2i and C2 = 1 + 4i.

ANSWER

1. Design overview: The class stores real and imaginary parts. Constructors initialize the object in different ways. Arithmetic operators return new Complex objects; equality compares both components.

2. Structured pseudocode:

```
CLASS Complex
PRIVATE:
  real : REAL
  imag : REAL

PUBLIC:
  CONSTRUCTOR()
    real <- 0.0
    imag <- 0.0
  ENDCONSTRUCTOR

  CONSTRUCTOR(r : REAL)
    real <- r
    imag <- 0.0
  ENDCONSTRUCTOR

  CONSTRUCTOR(r : REAL, i : REAL)
    real <- r
    imag <- i
  ENDCONSTRUCTOR

  OPERATOR +(REF other AS Complex) RETURNS Complex
    RETURN Complex(real + other.real, imag + other.imag)
  ENDOPERATOR

  OPERATOR *(REF other AS Complex) RETURNS Complex
    DECLARE newReal : REAL
    DECLARE newImag : REAL
    newReal <- real * other.real - imag * other.imag
    newImag <- real * other.imag + imag * other.real
    RETURN Complex(newReal, newImag)
  ENDOPERATOR

  OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
```

```
    RETURN (real = other.real AND imag = other.imag)
  ENDOPERATOR

  METHOD Display()
    DISPLAY real, " + ", imag, "i"
  ENDMETHOD
ENDCLASS

BEGIN
  DECLARE C1 : Complex <- Complex(3, 2)
  DECLARE C2 : Complex <- Complex(1, 4)
  DECLARE product : Complex
  product <- C1 * C2
  product.Display()
END
```

3. Dry-run trace: $(3 + 2i)(1 + 4i) = (3\times1 - 2\times4) + (3\times4 + 2\times1)i = -5 + 14i$.

Step	Calculation	Value
Real part	$3\times1 - 2\times4$	-5
Imaginary part	$3\times4 + 2\times1$	14
Product	real + imag i	-5 + 14i

Expected output:

```
-5 + 14i
```

Question 3: Financial Currency Class with Stream I/O & Unary Negation

Topic: Unary Operator Overloading, Friend Stream Operators (<<, >>).

Scenario: Design Currency with integer dollars and cents. Its constructor normalizes cents, unary minus reverses the balance sign, and friend stream operators read and display currency with two-digit cents.

Student Tasks: Write Currency and a driver. Trace d = 5, c = 275, its normalization, and unary negation.

ANSWER

1. Design overview: Convert the pair into a total number of cents, then normalize into dollars and cents. Unary negation returns a new Currency value with the opposite total amount. Stream operators are non-member friends so the stream is the left operand.

2. Structured pseudocode:

```
CLASS Currency
PRIVATE:
  dollars : INTEGER
  cents : INTEGER

  METHOD Normalize()
    DECLARE totalCents : INTEGER
    totalCents <- dollars * 100 + cents
    dollars <- totalCents DIV 100
    cents <- totalCents MOD 100
  ENDMETHOD

PUBLIC:
  CONSTRUCTOR(d : INTEGER = 0, c : INTEGER = 0)
    dollars <- d
    cents <- c
    CALL Normalize()
  ENDCONSTRUCTOR

  OPERATOR -() RETURNS Currency
    RETURN Currency(-dollars, -cents)
  ENDOPERATOR

  FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)
    DISPLAY "$", cur.dollars, ".", TWO_DIGIT(cur.cents)
    RETURN outStream
  ENDOPERATOR

  FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)
    INPUT cur.dollars, cur.cents
    CALL cur.Normalize()
    RETURN inStream
  ENDOPERATOR
ENDCLASS

BEGIN
  DECLARE cur : Currency <- Currency(5, 275)
  DISPLAY "Normalized: ", cur
  DECLARE debt : Currency
  debt <- -cur
  DISPLAY "Negated: ", debt
END
```

3. Dry-run trace: Input is 5 dollars and 275 cents. Total = 5×100 + 275 = 775 cents. Normalized amount is 7 dollars and 75 cents. Unary negation gives the opposite amount.

Step	Calculation	Dollars	Cents
Input	Given values	5	275
Normalize	775 DIV 100; 775 MOD 100	7	75
Unary minus	Opposite of +$7.75	-7	-75

Expected output:

```
Normalized: $7.75
Negated: $-7.75
```

Question 4: Polynomial Expression with Overloaded Subscript [] & Destructor

Topic: Dynamic Constructors, Destructors, Overloaded Subscript Operator [].

Scenario: Design Polynomial with a dynamic coefficient array of size degree + 1. The constructor initializes coefficients to zero, the copy constructor performs a deep copy, the destructor releases memory, [] returns a reference after bounds checking, and + adds coefficients of matching powers.

Student Tasks: Write Polynomial pseudocode. Trace $P1(x) = 3x^2 + 2x + 5$ and $P2(x) = 4x^3 + x^2 + 7$.

ANSWER

1. Design overview: Coefficient index represents the power of x. Addition creates a result whose degree is the larger input degree; missing higher coefficients are treated as zero. The subscript operator returns a reference to permit both reading and assignment.

2. Structured pseudocode:

```
CLASS Polynomial
PRIVATE:
  degree : INTEGER
  coeffArray : POINTER TO REAL

PUBLIC:
  CONSTRUCTOR(deg : INTEGER)
    degree <- deg
    coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
    FOR i <- 0 TO degree
```

```
    coeffArray[i] <- 0.0
  ENDFOR
 ENDCONSTRUCTOR

 COPY CONSTRUCTOR(REF other AS Polynomial)
  degree <- other.degree
  coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
  FOR i <- 0 TO degree
   coeffArray[i] <- other.coeffArray[i]
  ENDFOR
 ENDCONSTRUCTOR

 DESTRUCTOR()
  DEALLOCATE coeffArray
  coeffArray <- NULL
 ENDDESTRUCTOR

 OPERATOR [](index : INTEGER) RETURNS REF REAL
  IF index >= 0 AND index <= degree THEN
   RETURN REF coeffArray[index]
  ELSE
   DISPLAY "Coefficient index out of bounds"
   RETURN REF SAFE_ERROR_VALUE
  ENDIF
 ENDOPERATOR

 OPERATOR +(REF other AS Polynomial) RETURNS Polynomial
  DECLARE maxDegree : INTEGER
  maxDegree <- MAX(degree, other.degree)
  DECLARE result : Polynomial <- Polynomial(maxDegree)

  FOR i <- 0 TO maxDegree
   DECLARE leftValue : REAL <- 0
   DECLARE rightValue : REAL <- 0
   IF i <= degree THEN
    leftValue <- coeffArray[i]
   ENDIF
   IF i <= other.degree THEN
    rightValue <- other.coeffArray[i]
   ENDIF
   result.coeffArray[i] <- leftValue + rightValue
  ENDFOR
  RETURN result
 ENDOPERATOR
ENDCLASS
```

```
BEGIN
  DECLARE P1 : Polynomial <- Polynomial(2)
  P1[0] <- 5
  P1[1] <- 2
  P1[2] <- 3

  DECLARE P2 : Polynomial <- Polynomial(3)
  P2[0] <- 7
  P2[1] <- 0
  P2[2] <- 1
  P2[3] <- 4

  DECLARE P3 : Polynomial
  P3 <- P1 + P2
  DISPLAY P3
END
```

3. Dry-run trace: P1 coefficients by ascending power are [5, 2, 3, 0]. P2 coefficients are [7, 0, 1, 4]. Add corresponding coefficients.

Power	P1 coefficient	P2 coefficient	Sum coefficient
x^0	5	7	12
x^1	2	0	2
x^2	3	1	4
x^3	0	4	4

Expected polynomial:

```
P3(x) = 4x^3 + 4x^2 + 2x + 12
```

Question 5: 2D Vector with Prefix/Postfix Increment & Compound Assignment

Topic: Unary Operators (Prefix vs Postfix ++), Compound Assignment (+=).

Scenario: Design Vector2D with x and y coordinates. Prefix ++ increments both coordinates and returns the current object by reference. Postfix ++ returns an old-state copy by value after incrementing the object. += adds another vector's coordinates and returns the current object by reference.

Student Tasks: Write Vector2D and a driver. Trace v2 <- ++v1 and v3 <- v1++ when v1 begins at (5.0, 8.0).

ANSWER

1. Design overview: Prefix increment modifies the vector before returning it. Postfix increment stores the old vector, increments the original, and returns the stored copy. Compound assignment modifies the left-hand vector.

2. Structured pseudocode:

```
CLASS Vector2D
PRIVATE:
  x : REAL
  y : REAL

PUBLIC:
  CONSTRUCTOR(a : REAL = 0.0, b : REAL = 0.0)
    x <- a
    y <- b
  ENDCONSTRUCTOR

  OPERATOR ++() RETURNS REF Vector2D
    x <- x + 1.0
    y <- y + 1.0
    RETURN REF THIS
  ENDOPERATOR

  OPERATOR ++(dummy : INTEGER) RETURNS Vector2D
    DECLARE oldState : Vector2D
    oldState <- Vector2D(x, y)
    x <- x + 1.0
    y <- y + 1.0
    RETURN oldState
  ENDOPERATOR

  OPERATOR +=(REF other AS Vector2D) RETURNS REF Vector2D
    x <- x + other.x
    y <- y + other.y
    RETURN REF THIS
  ENDOPERATOR

  METHOD Display()
    DISPLAY "(", x, ", ", y, ")"
  ENDMETHOD
ENDCLASS

BEGIN
  DECLARE v1 : Vector2D <- Vector2D(5.0, 8.0)
  DECLARE v2 : Vector2D
```

```
  DECLARE v3 : Vector2D

  v2 <- ++v1
  v3 <- v1++
  DISPLAY "v1: "
  v1.Display()
  DISPLAY "v2: "
  v2.Display()
  DISPLAY "v3: "
  v3.Display()
END
```

3. Dry-run trace: Initially v1 = (5,8). Prefix increments v1 to (6,9), and v2 receives (6,9). Postfix saves (6,9) into v3, then increments v1 to (7,10).

Step	Operation	v1	v2	v3
Initial	—	(5,8)	—	—
1	v2 <- ++v1	(6,9)	(6,9)	—
2	v3 <- v1++	(7,10)	(6,9)	(6,9)

Expected output:

```
v1: (7, 10)
v2: (6, 9)
v3: (6, 9)
```

Important Notes

• The five debugging answers retain the source buggy code and provide corrected, complete C++ programs.

• Pseudocode follows the lifecycle and operator notation requested in Unit III.

• In Section B Question 4, the out-of-range subscript case is represented with a safe error reference; a real implementation should preferably use an exception or another explicitly defined error policy.

• Currency normalization is shown using total cents so values above 100 cents are carried into dollars.